Niveau 0 — Prise en main

> **Durée :** 1 h 30 · **Rendu :** un fichier `tp/reponses-niveau-0.md` + deux petites modifications de code

Avant de modifier une application, on la fait tourner et on la lit. Ce niveau ne demande presque pas de code : il demande de **comprendre** ce qui existe, parce que tous les niveaux suivants s'appuient dessus.

0.1 Lancer l'application

Suivez la section *Installation* du `README.md`. Vous avez terminé quand :

- ☐ http://localhost:8080/api/plats affiche du JSON ;
- ☐ http://localhost:5173 affiche l'ardoise avec les plats ;
- ☐ vous avez passé une commande depuis l'onglet **Prendre une commande** et la retrouvez dans l'onglet **Commandes**.

0.2 Lire le schéma relationnel

Ouvrez `database/01_schema.sql`.

1. Écrivez le schéma relationnel des 4 tables, en soulignant les clés primaires et en marquant les clés étrangères d'un `#` : `plat (id, nom, prix, #id_categorie)`…
2. Pourquoi `commande_ligne` contient-elle une colonne `prix_unitaire`, alors que le prix est déjà dans `plat` ? Donnez un exemple concret où les deux valeurs seraient différentes.
3. Que se passe-t-il pour les lignes d'une commande quand on supprime la commande ? Quelle instruction en est responsable ?
4. Repérez **trois données absurdes** que le schéma actuel accepte sans broncher (indice : essayez d'insérer à la main un prix négatif). Notez-les : vous les corrigerez au niveau 1 et 2.

0.3 Interroger l'API à la main

Avec `curl` (ou Thunder Client), exécutez et notez **le code HTTP** et **la réponse** de chaque requête :

```
# a) liste des soupes
curl http://localhost:8080/api/plats?categorie=2

# b) création d'un plat valide
curl -X POST http://localhost:8080/api/plats \
     -H "Content-Type: application/json" \
     -d '{"nom":"Mofo gasy","prix":300,"id_categorie":1}'

# c) création d'un plat invalide
curl -X POST http://localhost:8080/api/plats \
     -H "Content-Type: application/json" \
     -d '{"nom":"","prix":"cher"}'
```

```
# d) suppression d'un plat présent dans une commande
curl -X DELETE http://localhost:8080/api/plats/2

# e) une commande qui n'existe pas
curl http://localhost:8080/api/commandes/999
```

Ajoutez `-i` à la commande `curl` pour voir le code HTTP.

5. Pour la requête **b**, quel code obtenez-vous ? Pourquoi pas 200 ?
6. Pour la requête **d**, qui bloque la suppression : PHP ou PostgreSQL ? Montrez la ligne de code qui transforme ce blocage en réponse 409.
7. Créez une commande avec `curl` (table 6, deux *Romazava* et un *Café*). Écrivez la commande `curl` complète.

0.4 Suivre une requête de bout en bout

Quand on clique sur **Ajouter le plat**, voici le chemin parcouru. Complétez le tableau en indiquant, pour chaque étape, **le fichier et le nom de la fonction** :

#	Étape	Fichier	Fonction / ligne
1	Le formulaire est soumis	`components/PlatForm.vue`	
2	La requête HTTP est construite		`requete()`
3	Vite transmet `/api/...` au port 8080		
4	CodeIgniter choisit le contrôleur		
5	Le contrôleur lit le JSON		
6	Les règles de validation sont vérifiées		
7	La ligne est insérée		
8	Le formulaire prévient l'écran parent		`emit(…)`
9	L'écran recharge la liste	`views/CarteView.vue`	

8. Où est le code métier de la création d'une commande ? Pourquoi n'est-il pas dans le contrôleur `Commandes.php` ?

0.5 Repérer les notions de Vue

Vue est nouveau pour vous. Le code de base utilise sept notions ; trouvez **un exemple de chacune** (fichier + ligne) et expliquez-le en une phrase.

Notion	À quoi ça sert	Votre exemple
`ref()`	une variable dont le changement met l'écran à jour	

Notion	À quoi ça sert	Votre exemple
`computed()`	une valeur calculée à partir d'autres, recalculée automatiquement	
`v-for`	répéter un élément pour chaque élément d'une liste	
`v-if` / `v-else`	afficher un élément seulement si une condition est vraie	
`v-model`	lier un champ de formulaire à une variable, dans les deux sens	
`defineProps`	recevoir des données du composant parent	
`defineEmits` / `emit`	envoyer un événement au composant parent	
`onMounted`	exécuter du code quand le composant apparaît	

9. `Panier.vue` ne modifie jamais la quantité lui-même : il émet `plus` et `moins`. Pourquoi ce choix ? Qui modifie réellement la quantité ?
10. Dans `PlatForm.vue`, à quoi sert le `watch` ? Que se passerait-il sans lui quand on clique sur **Modifier** d'un autre plat ?

0.6 Premières modifications

Deux petites modifications pour vérifier que tout est en place.

a) Le nom de la gargote. Remplacez « Chez Bao, Analakely » par le nom de votre choix. Un seul fichier à modifier.

b) Une nouvelle catégorie, sans toucher au code. Ajoutez la catégorie *Goûter* directement en SQL, puis rechargez l'interface.

```
INSERT INTO categorie (libelle) VALUES ('Goûter');
```

11. La catégorie apparaît dans la liste déroulante du formulaire sans que vous ayez écrit de code. Expliquez pourquoi, en citant la route et le composant concernés.

Validation du niveau 0

- ☐ L'application tourne (API + interface).
- ☐ `reponses-niveau-0.md` répond aux 11 questions et contient le tableau 0.4 complet.
- ☐ Le nom de la gargote est modifié ; *Goûter* existe en base.
- ☐ Un commit `niveau 0 : prise en main`.